



《微机原理与接口技术》课程设计报告

题目 实验 1：变电站自动监测系统

姓 名：田佳铭

学 号：22920172204211

院 系：信息学院人工智能系

专 业：智能科学与技术

年 级：2017 级

指导教师：林贤明

2020 年 6 月 13 日

目录

一、实验名称与描述.....	1
二、实验环境与文件说明.....	1
三、整体思路.....	2
四、具体芯片设计.....	2
4.1 接收端芯片.....	2
4.1.1 74LS138.....	2
4.1.2 ADC0809.....	3
4.1.3 8255A.....	3
4.1.4 8259A.....	3
4.1.5 8251A.....	3
4.1.6 8253.....	4
4.2 发送端芯片.....	5
4.2.1 74LS138.....	5
4.2.2 8259A.....	5
4.2.3 8251A.....	5
4.2.4 8253.....	5
4.3 各芯片设置的汇总表格.....	6
五、部分代码设计.....	8
5.1 发送端代码流程.....	8
5.1.1 从 ADC0809 中读取数据.....	8
5.1.2 将数据发送给 8251A.....	9
5.2 接收端代码流程.....	9
5.2.1 从 8251A 读取温度数据.....	10
5.2.2 对温度信息进行处理.....	10
5.3 一些具体的代码实现思路.....	11
5.3.1 输出 10 进制数.....	11
5.3.2 从 ADC0809 读取的数据最终转换成温度值.....	11
5.4 附加功能.....	11

5.4.1 芯片地址快速更改.....	12
5.4.2 温度信息比较.....	13
5.4.3 接收端信息检验.....	13
六、实验收获与感想.....	14

一、实验名称与描述

实验 1：变电站自动监测系统。

某无人变电站需要 24 小时监测变电站中各电气装置的温度，并将采集到的信息利用串行通信的方式通过 modem 发到远端控制中心，控制中心的管理系统则接收来自无人变电站采集的信息，并在监视屏上实时显示，一旦出现不正常数据，就立即给出警报信息，提醒值班人员进行应急处理。假设现在需要采集的温度信息有 8 路，这些信息每 10ms 采集一次，温度信息可以通过温度传感器转化为模拟电信号。假设温度变化范围是 $0 \sim 100^{\circ}\text{C}$ ，对应的传感器输出电压信息为： $0\text{V} \sim 10\text{V}$ 。正常温度时，监测中心显示温度值，一旦发现温度超过 60°C ，则在屏幕上显示告警信息，用以提醒值班人员。

系统实现需要的芯片请自行选择，芯片使用的端口地址也自行设定，并给出相应的说明，请根据题目设计要求完成该无人变电站自动监测系统。

二、实验环境与文件说明

本次实验的代码部分主要使用 emu8086 工具，这是模拟 8086CPU 运行的一个模拟器，但是仿真功能有限，不能完全仿真各种芯片的功能。还有一些需要注意的是，emu8086 不支持一些真正的汇编语言支持的内容，例如它好像不支持变量+`DUP(?)`的写法，这样的地方在代码中有所标注，还有，emu8086 的代码好像不支持代码段分段，虽然中断子程序有 FAR 声明，但是移植到其他平台依然可能会出现一些问题。

文件夹“代码”中“send”为发送端代码，“receive”为接收端代码，均分为“*.asm”格式和“*.txt”格式。

文件夹“硬件电路图”中是本设计的硬件连接示意图，分为 pdf 和照片格式两种。

三、整体思路

整个硬件电路分为两部分，即发送端和接收端。发送端是在无人变电站中，主要完成的作用是采集的温度信息和将信息发送给接收端。接收端的作用是从发送端接受数据并且将信息打印在屏幕上，同时进行“给出警报信息”等相关操作。发送端和接收端通过异步 modem 连接。

发送端的主要思路是：使用 CPU 的时钟（8MHz），并且用 8253 来产生各种需要的时钟信号（ADC0809 的时钟信号和 8251A 的时钟信号），同时每 10ms 产生一个正跳变脉冲用以使 8259A 产生中断信号。在中断子程序中，CPU 通过 8255A 从 ADC0809 中读取数值，并通过 8251A 将数据发送给 modem。

接收端也是使用 8253 来产生 8251A 需要的时钟信号，并且通过 8251A 的 RxRDY 使 8259A 产生中断信号。在中断子程序中，CPU 主要从 8251A 中读取数据并根据书瑶进行数据处理。

两端均使用 8086CPU，并且使用 74LS138 来做芯片选择。

四、具体芯片设计

本部分将芯片分为两大部分，即发送端芯片和接收端芯片。具体的硬件连接请看硬件连接图。

4.1 接收端芯片

4.1.1 74LS138

接收端使用 74LS138 来做芯片选择。芯片地址分布如下（均使用连续地址）：

芯片	地址范围
8255A	300H-303H
8253	304H-307H
8259A	308H-309H
8251A	312H-313H

4.1.2 ADC0809

我们用 8255A 来读取 ADC0809 的数据（两者之间的连接在 8255A 那里介绍）。

ADC0809 与环境的连接方式为 IN7~IN0 连接 8 路的传感器和运放（如果需要的话），用来接收来自环境的模拟信号。ADC0809 的时钟使用从 8253 来的 1MHz 的方波信号。

ADC0809 不需要初始化编程。

4.1.3 8255A

8255A 主要用于读取 ADC0809 的数据。

ADC0809 的输出接在 8255A 的 A 口，将 A 口编程为方式 0，输入方式。C 口的高四位编程为输入，低四位为输出。ADC 的 START 和 ALE 与 PC3 相连，由 CPU 控制 PC3 发启动信号和通道锁存信号，PC2-PC0 输出三位通道号地址信号。EOC 输出信号和 EOC 为低电平，表示转换已开始。然后继续查询，当发现 EOC 变高，便说明转换已结束。由于 EOC 还和输出使能端 OE 相连，转换结束时 OE 也变高，使 ADC 的输出缓冲器打开，数据出现在 A 口上，可由 IN 指令读入 CPU。

4.1.4 8259A

8259A 主要用于产生中断信号。因为需要每 10ms 采集一次数据，所以这里使用 8259A 的 IR2 端口用来产生中断。（IR1 用做键盘中断，除 IR1 和 IR2 外其他中断请求信号输入引脚应该被屏蔽）

8259 的具体设置为：单片 8259A（只需要两个中断信号）、全嵌套方式（需要屏蔽同级中断信号）、边沿触发（配合 8253 的脉冲信号）、普通 EOI 方式、中断类型号基值为 50H。

4.1.5 8251A

8251 主要用于将要发送的数据转换为串行信号并发送给 modem。这里设置

发送的波特率 9600Bd、波特率系数 16（注意这里要和接收端的设置相同）。

其他的一些初始化设置为：1 位停止位、偶校验（这两个值的设置可以比较随意，主要是要和接收端相同）、数据位 8 位（因为我们从 ADC0809 采集到的信息是 8 位的）。

4.1.6 8253

8253 的作用是产生所需要的各种信号。需要使用到信号的芯片有三个：ADC0809 需要的时钟信号、8251A 需要的时钟信号和 8259A 需要的正跳变脉冲信号。

三者所需要的信号的频率不一样，这样刚好使用到了三个通道，不再需要第二块 8253。

三个通道的设置如下：

（1）通道 0。通道 0 使用 8MHz 的 CPU 时钟，用来给 ADC0809 提供时钟，时钟频率为 1MHz。因此需要通道 0 工作在方式 3。

可以计算出时间常数= $8\text{MHz}/1\text{MHz}=8$ 。

可以得出通道 0 的具体设置为：方式 3，先写低字节，后写高字节，使用 BCD 码，时间常数为 8。

（2）通道 1。通道 1 使用 8MHz 的 CPU 时钟，用来给 8251A 提供时钟。因此需要通道 1 工作在方式 3。

已知串行数据传送的波特率被设置为 9600Bd，波特率系数为 16，则 $\overline{RxC}$ 和 $\overline{TxC}$ 的频率为： $9600*16=153600\text{Hz}=0.1536\text{MHz}$ 。可以计算出时间常数= $8\text{MHz}/0.1536\text{MHz}=52$ 。

可以得出通道 1 的具体设置为：方式 3，先写低字节，后写高字节，使用 BCD 码，时间常数为 52。

（3）通道 2。通道 2 使用 8253OUT1 的时钟：0.1536MHz（考虑到直接使用 8MHz 的 CPU 时钟或者通道 0 的 1MHz 的时钟会使得时间常数太大），用来给发送端 8259A 提供中断脉冲，频率 100MHz。因此需要通道 2 工作在方式 2。

时间常数= $0.1536\text{MHz}/100\text{Hz}=1536$

可以得出通道 1 的具体设置为：方式 2，先写低字节，后写高字节，使用

BCD 码，时间常数为 1536。

4.2 发送端芯片

4.2.1 74LS138

发送端使用 74LS138 来做芯片选择。芯片地址分布如下（均使用连续地址）：

芯片	地址范围
8253	304H-307H
8259A	308H-309H
8251A	312H-313H

4.2.2 8259A

这里使用 8259A 的 IR2 端口用来产生中断。（IR1 用做键盘中断，除 IR1 和 IR2 外其他中断请求信号输入引脚应该被屏蔽）

8259 的具体设置为：单片 8259A（只需要两个中断信号）、全嵌套方式（需要屏蔽同级中断信号）、边沿触发（配合 8253 的脉冲信号）、普通 EOI 方式、中断类型号基值为 50H。

4.2.3 8251A

8251 主要用于从 modem 接收数据并将数据转换成串行数据。这里设置接收的波特率 9600Bd、波特率系数 16（注意这里要和发送端的设置相同）。

其他的一些初始化设置为：1 位停止位、偶校验（这两个值的设置可以比较随意，主要是要和发送端相同）、数据位 8 位。

比较关键的一点是，8251A 的 RxRDY 端要接 8259A 的 IR2。

4.2.4 8253

接收端的 8253 只需要用来给 8251A 提供时钟。这里使用通道 1，输入信号使用 8MHz 的 CPU 信号。基本设置和发送端的 8253 的通道 1 基本相同（发送端和接收端的 8251A 的波特率和波特率常数相同）。可以得出通道 1 的具体设置

为：方式 3，先写低字节，后写高字节，使用 BCD 码，时间常数为 52。

4.3 各芯片设置的汇总表格

为了使各芯片的设置更加清楚，我们可以汇总出如下的表格（这里只列出有地址、需要初始化的一些芯片，74LS138 和 ADC0809 暂且不列出）：

芯片	地址	初始化设置	方式字/命令字/控制字等
发送端-8255A	300H-303H A 口：300H B 口：301H C 口：302H 控制口：303H	(1) A 口编程为方式 0 (2) C 口的高 4 位编程为输入，低 4 位为输出	控制字： 10011000B (B 口相关设置为默认全 0)
发送端-8253	304H-307H 通道 0：304H 通道 1：305H 通道 2：306H 控制口：307H	(1) 通道 0：方式 3，先写低字节，后写高字节，BCD，时间常数 8 (2) 通道 1：方式 3，先写低字节，后写高字节，BCD，时间常数 52 (3) 通道 2：方式 2，先写低字节，后写高字节，BCD，时间常数 1536	(1) 通道 0 控制字： 00110111B (2) 通道 1 控制字： 01110111B (3) 通道 2 控制字： 10110101B
发送端-8259A	308H-309H	(1) 单片 8259A，边沿触发 (2) 中断类型号 50H-57H (3) 全嵌套，普通 EOI (4) 不屏蔽键盘中断 (IR1) 和使用到的 IR2	初始化命令字： (1) ICW1： 00010011B (2) ICW2： 50H (3) ICW4： 01H (00000001B) 操作命令字：

			OCW1: 11111001B
发送端-8251A	312H-313H 数据口: 312H 控制口: 313H	(1) 串行异步通信、波特率 9600Bd、波特率系数 16 (2) 1 位停止位、偶校验、数据位 8 位 (3) 需要清楚错误标志、允许发送、屏蔽接收	(1) 输出方式字: 01111110B (2) 输出命令字: 00010001B (还需要复位, 即先向控制口写 3 个 0, 再送复位字 40H)
接收端-8253	304H-307H 通道 0: 304H 通道 1: 305H 通道 2: 306H 控制口: 307H	(2) 通道 1: 方式 3, 先写低字节, 后写高字节, BCD, 时间常数 52	(2) 通道 1 控制字: 01110111B
接收端-8259A	308H-309H	(1) 单片 8259A, 边沿触发 (2) 中断类型号 50H-57H (3) 全嵌套, 普通 EOI (4) 不屏蔽键盘中断 (IR1) 和使用到的 IR2	初始化命令字: (1) ICW1: 00010011B (2) ICW2: 50H (3) ICW4: 01H (00000001B) 操作命令字: OCW1: 11111001B
接收端-8251A	312H-313H 数据口: 312H 控制口: 313H	(1) 串行异步通信、波特率 9600Bd、波特率系数 16 (2) 1 位停止位、偶校验、数据位 8 位	(1) 输出方式字: 01111110B (2) 输出命令字: 00010100B (还需要复位, 即

		(3) 需要清楚错误标志、 允许接收、屏蔽发送	先向控制口写 3 个 0,再送复位字 40H)
--	--	----------------------------	----------------------------

五、部分代码设计

这里主要是一些重要代码的实现，例如初始化写入方式字/命令字/控制字等代码不再做具体的描述（因为在第四部分最后的表格中已经给出地址和方式字/命令字/控制字的值，实现起来就很清楚了）。这里主要讲大体实现思路，基本不涉及具体的代码（具体实现请参照源代码）。

5.1 发送端代码流程

发送端代码的主要流程是：

(1) 建立中断向量（主要是 IR2 的中断向量设置，这里暂且不考虑键盘中断的中断向量设置）

(2) 8255A 初始化编程

(3) 8253 初始编程

(4) 8259A 初始化编程

(5) 8251A 初始化编程

(6) 等待中断（这里用了忙等的方式，在实际使用中，CPU 这时还可以做其他的一些工作）

中断子程序的主要流程是：

(1) 从 ADC0809 中读取数据

(2) 将数据发送给 8251A

这两个部分的实现思路为：

5.1.1 从 ADC0809 中读取数据

从 ADC0809 中读取数据是通过 8255A 实现的。

程序是一个循环，分别读取 8 个通路。对于每个通路，步骤为：

- (1) 预处理（通道号+1 等，当然，也可以再循环末尾做）
- (2) 发送通道号
- (3) 置 PC3=1（在 START 和 ALE 脚上加上一个正脉冲，将通道选择码锁存并启动 AD 转换）发送
- (4) PC3 复位，停止转换（这之前需要 NOP 延时）
- (5) 检查 PC7，一直到 PC7=0，表示已经启动转换（即查看 EOC 信号）
- (6) 检查 PC7，一直到 PC7=1，表示已经转换结束
- (7) 从 A 口读入数据
- (8) 存储数据

5.1.2 将数据发送给 8251A

读取完所有的数据后，一起发送。对每个数据，发送流程是：

- (1) 预处理（数据地址偏移+1 等，当然，也可以再循环末尾做）
- (2) 检查状态字 D0，直至 D0=1（发送器准备好）
- (3) 从内存中取出要发送的数据
- (4) 将数据发送给 8255A

5.2 接收端代码流程

发送端代码的主要流程是：

- (1) 建立中断向量（主要是 IR2 的中断向量设置，这里暂且不考虑键盘中断的中断向量设置）
- (2) 8253 初始编程
- (3) 8259A 初始化编程
- (4) 8251A 初始化编程
- (5) 等待中断（这里用了忙等的方式，在实际使用中，CPU 这时还可以做其他的一些工作）

中断子程序的主要流程是：

- (1) 从 8251A 读取温度数据
- (2) 对温度信息进行处理

这两个部分的实现思路为：

5.2.1 从 8251A 读取温度数据

从发送端的程序可以看到，发送端一旦开始发送数据，就会将 8 组数据连续发送。所以接收时，也是将所有数据一起接受。

共接收 8 个数据，每个数据的接收流程是：

- (1) 预处理（数据地址偏移+1 等，当然，也可以再循环末尾做）
- (2) 检查状态字 D1，直至 D0=1（接收器准备好）（因为是将 RxRDY 作为中断控制信号，第一个数据也可以不用检验，因为是采用普通 EOI 方式，所以也不需要担心 RxRDY 信号会产生新的中断）

(3) 接收数据

(4) 存储数据

这里可以做一些错误处理，例如在接收数据前，查看状态字，看是否有相应错误位为 1，如果有，这里采取比较简单的做法，即直接结束中断（当然，结束中断时候还需要一些处理，例如开中断、恢复现场等）。在代码中，错误处理部分是注释掉的，即默认不会出现数据传输错误，因为如果要这样做的话，还需要一些其他的机制（见 5.4.3）。

5.2.2 对温度信息进行处理

接收完数据后，就可以对数据进行处理。

对每个温度信息的处理过程是：

- (1) 预处理（数据地址偏移+1 等，当然，也可以再循环末尾做）
- (2) 从内存取出数据
- (3) 将数据转换为温度值
- (4) 将相关字符串和温度值（10 进制形式）打印出来
- (5) 将温度信息和 60（10 进制）做对比，如果大于或等于 60 度（安全起见，60 度也要打印温度信息），则打印警告信息

5.3 一些具体的代码实现思路

这里介绍一些具体的代码实现思路，这里主要是一些基本功能的实现。

5.3.1 输出 10 进制数

我们输出的数据是按 10 进制显示的，所以需要有一个输出 10 进制的函数。

函数采用比较常见的除以 10 和压栈的方式来实现。因为我们的数据是无符号数，所以实现比较简单。实现方式如下：

- (1) 将 10 压入栈
- (2) 数据除以 10，将余数压栈
- (3) 如果商为 0（说明除尽），则转到（5）
- (4) 转到（2）
- (5) 弹栈
- (6) 如果弹出的数据是 10（表示打印已经完毕），则转到（10）
- (7) 将数据加 30H（转换成对应的 ASCII 码）
- (8) 以 ASCII 形式打印数据
- (9) 转到（5）
- (10) 结束程序

5.3.2 从 ADC0809 读取的数据最终转换成温度值

我们要做的是将 ADC0809 的数据转换成对应的温度信息。温度变化范围是 0~100℃，对应我们的数据 0-255。因此只需要将数据除以 2.56 即可。这里不考虑浮点数运算，所以实现方式是数据先乘 25 再除以 64，并且采用向上取整的方式（为了安全起见，即使是接近 60 度但是还没到 60 度也选择警告）。

5.4 附加功能

除了基本的功能，本设计还加入了其他的一些附加功能，需要注意的是，在具体实现中，附加功能和上述的整体处理流程是混合在一起的。

5.4.1 芯片地址快速更改

考虑到实际情况中，芯片地址可能会更换，为了在新片地址改变时可以快速修改代码，这里使用了两个机制来做新片地址快速更改。

(1) 将芯片地址设为变量。

如图所示：

```
P_8255A DW 300H,301H,302H,303H ;8255A通道地址
P_8253 DW 304H,305H,306H,307H ;8253通道地址
P_8259A DW 308H,309H ;8259A通道地址
P_8251A DW 312H,313H ;8251A通道地址
```

图 5-1 芯片地址变量（以发送端为例）

这样的话就可以通过变量名来使用地址信息，更改地址的时候就可以很方便地进行修改。

(2) 修改芯片地址的宏

虽然有了机制（1），但是这样仍然需要修改 12 个值（以发送端为例），为了更加方便，这里使用宏来做更加快速的修改。

最原始的宏接受 5 个参数，分别是：

(a) CHIP：芯片对应的起始地址的变量名

(b) ADDR：起始地址

(c) TYPE：地址类型。01H 为连续地址，02H 为基地址

(d) NUM：需要更改的地址数，为字类型。根据地址变量数组长度来确定

(e) CH_ADDR：循环名，因为宏是直接展开，而在这个宏中使用了循环语句并且需要多次调用它，所以需要用它来防止出错。

这个宏比较简单，只需要即根据 NUM 初始化 CX 的值，用 ADDR 初始化 AX 的值，通过 CHIP 初始化 DI 的值（通过 LEA 命令），每次循环后 AX+TYPE 即可（这也是 TYPE 这样设计的目的）。

在此基础上我们可以再写三个宏：

(1) 修改四个芯片地址的宏（以发送端为例，接收端类似）。需要八个参数，即四个芯片的起始地址和地址类型，通过调用四次最原始的宏来实现。

(2) 修改四个芯片地址的宏，全部连续地址。只需要四个参数，即四个芯片的起始地址。

(3) 修改四个芯片地址的宏，全部为奇地址或者偶地址。只需要四个参数，

即四个芯片的起始地址。

5.4.2 温度信息比较

除了最简单的直接打印信息之外，此程序还增加了温度信息比较和输出温度变化趋势的代码，为了简单起见，温度只和上次传输来的信息进行比较。

实现起来比较简单，多增加一个用来储存旧温度信息的数组和存储比较信息的数组（主要因为这里在接收数据的时候进行比较，所以这个信息也需要储存）。

接收数据后，先将数据和原温度信息进行比较，将比较信息写入比较信息数组中，并在打印完温度信息后打印温度变化情况（升高、降低、不变）。在中断结束之前还需要将新温度信息写入旧温度信息。（虽然这里也可以不用存储旧温度信息的数组，即获得新数据后直接进行比较后再写入即可，但为了代码更加清晰，还是保留这个数组，而且，这个数组也只占 8 个字节，不会太过浪费空间）

这里还有一点需要注意的是，第一次接收到的温度信息是不需要进行比较的。（要做到这一点也很简单，设置一个变量，从 0 开始，打印温度信息时这个数值就加 1，直到加到 8 就停止增加，并且在这之后才会开始打印温度比较信息）

5.4.3 接收端信息检验

如果不添加接收端信息检验的话，如果信息出错（发送端的 8251A 状态字中相关错误位为 1），一般情况是丢掉数据，这可能会导致下一次接收时不知道当前是哪个通道的数据。于是，我决定添加信息检验。（当然，这里假设接收数据错误率很低）

这里需要对发送端发送的数据做一些处理，发送端不能只发送每个通道采集到的温度信息，还需要发送通道号。发送顺序设定为 0（通道 0）-通道 0 数据-1（表示通道 1）-...-7（表示通道 7）-通道 7 数据，即需要一个中断中发送 16 次数据。

接收端程序也需要相应的修改，主要是要利用好通道号信息。在进入中断的时候，将接收到的第奇数个数据和当前期望通道数进行比较，只有两者相同的情况下才会接受第偶数个数据，16 个数据全部接受无误的话才会进行温度打印的工作。因为接收端的中断控制信号是 RxRDY 信号，所以不需要担心数据中途出

错的情况，而如果接收端采用 10ms 一中断的情况就会出现麻烦。

六、实验收获与感想

其实做这个实验的时候，我的想法还是很多的。最早的一个想法是，我可以不可以不在变电站端放置一个 CPU，而只需要用到控制中心的管理系统中的 CPU？例如，我们是否可以用数块 8251A 来作为“延长”CPU 系统总线的手段，后来发现，8251A 也是需要先初始化才可以使用的，与其先在控制中心初始化后再送去变电站端，还不如直接在变电站端放一个 CPU，而且，如果真的用这种方式来“延长”系统总线，就会使得硬件图与控制代码非常复杂（特别是对于控制总线，我们需要根据具体的芯片单独考虑每一位），因而我放弃了这种想法。第二个想法是，我是不是可以在控制中心的管理系统发现温度过高的时候再发送信息回变电站段并显示出来呢？后来想到，这是个无人变电站，所以，即使我这样做了，变电站段的信息也没人注意得到，甚至有可能影响变电站段系统的正常工作。

在写发送端数据的时候，我也想着，既然检查 EOC 信号需要循环很多次，发送数据也需要等待，是否可以在这个时间段来发送数据，但是为了代码的清晰（如果这样做，代码会杂糅在一起），还是决定先采集所有信号后再统一发送。在将通道号也发送后，我也有考虑是否可以发送采集的时间（因为数据传输总是有延时的），但是汇编读取时间的代码会比较长，并且，根据设计，还需要对时间设置单独的检测，会使得代码过于复杂，也就没有添加。

虽然这样，但我还是想到了一些办法来使我的系统更完善一些，例如错误检验和温度比较。在整个实验的过程中我也学到了很多，复习到了很多课堂上没怎么注意到的小细节，同时也训练了我的汇编能力，例如，我使用了宏来使程序更加简洁一些。整个实验做下来感觉还是很好的。